

Resenha - Facade

Fonte: Refactoring.Guru

Autor da Resenha: Felipe Luiz Parreiras Lima

Introdução

O padrão de projeto Facade (Fachada) é um padrão estrutural que tem como principal objetivo simplificar a interação com sistemas complexos. Em muitos sistemas de software, especialmente os de maior porte, é comum que existam diversas classes interdependentes, com fluxos de execução específicos e regras que precisam ser seguidas corretamente. Esse cenário pode tornar o uso do sistema difícil e aumentar o acoplamento entre os componentes.

Nesse contexto, o Facade surge como uma solução para fornecer uma interface mais simples e organizada, permitindo que o cliente utilize o sistema sem precisar conhecer seus detalhes internos. Dessa forma, o padrão contribui para melhorar a legibilidade, a manutenção e a organização do código.

Ideia central do padrão

A ideia central do Facade é ocultar a complexidade interna de um subsistema por meio de uma interface única e de alto nível. Em vez de o cliente precisar interagir diretamente com várias classes, ele passa a utilizar apenas a fachada, que se responsabiliza por coordenar as chamadas necessárias.

Isso significa que o cliente não precisa conhecer a ordem das operações, as dependências entre objetos ou as regras específicas de execução. A Facade atua como um intermediário, delegando as responsabilidades para os componentes corretos.

É importante destacar que o padrão não modifica a lógica do sistema, mas apenas reorganiza o acesso a ele. As classes do subsistema continuam existindo e podem ser utilizadas diretamente, se necessário, mas a Facade oferece uma forma mais simples e segura de interação.

Aplicação em projetos reais

Na prática, o padrão Facade é amplamente utilizado em sistemas que possuem múltiplas camadas ou integrações, como aplicações web, APIs e sistemas corporativos. Um exemplo comum é a criação de serviços que centralizam chamadas a diferentes endpoints e realizam o tratamento dos dados antes de retorná-los ao cliente.

Em aplicações frontend, por exemplo, é comum utilizar uma camada intermediária que encapsula chamadas de API, regras de negócio e formatação de dados. Dessa forma, os componentes da interface não precisam conhecer detalhes como endpoints, estruturas de resposta ou transformações necessárias.

Além disso, o Facade também é útil em integrações com sistemas externos ou legados, onde a complexidade pode ser ainda maior. Ao utilizar uma fachada, é possível padronizar o acesso e reduzir o impacto de mudanças nesses sistemas.

Outro ponto relevante é a melhoria na manutenção do código. Como o cliente depende apenas da Facade, alterações internas no subsistema tendem a ter menos impacto nas demais partes do sistema, reduzindo o acoplamento e facilitando a evolução da aplicação.

Conclusão

O padrão Facade é uma solução eficaz para lidar com a complexidade de sistemas de software, oferecendo uma interface simplificada para o cliente. Ele contribui para a redução do acoplamento, melhora a organização do código e facilita a manutenção.

Embora não elimine a complexidade interna do sistema, o padrão torna essa complexidade mais controlada e menos exposta. No entanto, é importante utilizá-lo com equilíbrio, evitando que a Facade concentre responsabilidades excessivas e se torne difícil de manter.

De modo geral, o Facade é um padrão bastante útil em projetos reais, especialmente em sistemas maiores, onde a simplicidade na interação com os componentes é um fator essencial para a qualidade do software.